

MODULE 9

Matplotlib

Topics Covered

- Lecture 1 — Matplotlib Overview & Installation
- Lecture 2 — Getting Started Practically with Matplotlib
 - Lecture 3 — Deep Dive into Line Plot
 - Lecture 4 — Scatter Plot
 - Lecture 5 — Getting Started with Bar Plot
- Lecture 6 — Horizontal, Multiple & Stacked Bars
 - Lecture 7 — Histogram
- Lecture 8 — Pie Chart & Changing Styles

Lecture 1: Matplotlib Overview & Installation

Matplotlib is Python's most widely used data visualisation library. It produces publication-quality plots in a variety of formats and can be embedded in applications or Jupyter notebooks.

□ Why Matplotlib?

It is the foundation of other plotting libraries like Seaborn and Pandas .plot().
Full control over every element of a figure — fonts, colours, axes, ticks, gridlines.
Works in scripts, Jupyter notebooks, and GUI applications.

Installation & Import

```
pip install matplotlib

import matplotlib.pyplot as plt  # 'plt' is the universal alias
import numpy as np

# In Jupyter Notebook — render plots inline
%matplotlib inline
```

Matplotlib Architecture

Layer	Role
Figure	The top-level container — the entire window or canvas
Axes	An individual plot area inside the Figure (has x-axis, y-axis, title)
Artist	Everything drawn on the canvas: lines, text, patches, ticks, labels

□ Figure vs Axes

One Figure can contain multiple Axes (subplots).
plt.figure() creates the Figure. plt.axes() or plt.subplots() creates the Axes.
Most plt.xxx() calls act on the CURRENT active Axes automatically.

Two Ways to Plot

Style	When to Use
pyplot (plt.plot)	Quick, interactive — great for simple single plots
OO (fig, ax = plt.subplots())	Recommended for multi-plot figures and customisation

Lecture 2: Getting Started Practically with Matplotlib

Your First Plot

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 6, 3]

plt.plot(x, y)
plt.show()          # display the plot
```

Essential Labelling

```
plt.plot(x, y)

plt.title('My First Plot')      # chart title
plt.xlabel('X Axis Label')     # x-axis label
plt.ylabel('Y Axis Label')     # y-axis label
plt.legend(['Series 1'])       # legend
plt.grid(True)                 # show gridlines

plt.show()
```

Figure Size & Saving

```
plt.figure(figsize=(10, 5))    # width=10, height=5 inches
plt.plot(x, y)
plt.title('Sized Plot')

plt.savefig('my_plot.png', dpi=150, bbox_inches='tight')
plt.show()

# Supported formats: .png .jpg .pdf .svg
```

Subplots — Multiple Plots in One Figure

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 8))

axes[0,0].plot([1,2,3], [4,5,6])
axes[0,0].set_title('Top Left')

axes[0,1].scatter([1,2,3], [6,2,8])
axes[0,1].set_title('Top Right')

axes[1,0].bar(['A','B','C'], [3,7,5])
axes[1,0].set_title('Bottom Left')

axes[1,1].hist([1,2,2,3,3,3,4,4,4,4])
axes[1,1].set_title('Bottom Right')

plt.tight_layout()    # prevent overlap
plt.show()
```

Function	Description
<code>plt.figure(figsize)</code>	Create figure with given size (inches)

Function	Description
<code>plt.subplots(r, c)</code>	Create figure + grid of r×c axes
<code>plt.tight_layout()</code>	Auto-adjust spacing between subplots
<code>plt.savefig(path)</code>	Save figure to file
<code>plt.clf()</code>	Clear current figure
<code>plt.close()</code>	Close current figure window

Lecture 3: Deep Dive into Line Plot

Line plots are ideal for showing trends over time or ordered categories. `plt.plot()` is the most versatile function in Matplotlib.

Line Styling

```
import numpy as np
x = np.linspace(0, 2 * np.pi, 100)

plt.plot(x, np.sin(x), color='blue', linestyle='-', linewidth=2, label='sin(x)')
plt.plot(x, np.cos(x), color='red', linestyle='--', linewidth=2, label='cos(x)')
plt.plot(x, np.tan(x), color='green', linestyle=':', linewidth=1, label='tan(x)')

plt.title('Trigonometric Functions')
plt.xlabel('Angle (radians)')
plt.ylabel('Value')
plt.ylim(-2, 2)      # limit y-axis range
plt.legend()
plt.grid(True)
plt.show()
```

Line Style & Marker Reference

Parameter	Common Values
color	'blue' 'red' '#FF5733' (0.1, 0.2, 0.5) — RGB tuple
linestyle	'-' solid '--' dashed ':' dotted '-.' dash-dot
linewidth	Float — e.g. 1.5, 2.0, 3.0
marker	'o' circle 's' square '^' triangle '*' star 'D' diamond '.' point
markersize	Float — size of the marker
alpha	0.0 (transparent) to 1.0 (opaque) — controls transparency
label	String shown in legend

Axis Customisation

```
plt.plot(x, y)

plt.xlim(0, 10)      # set x-axis range
plt.ylim(-5, 5)      # set y-axis range
plt.xticks([0, 2, 4, 6, 8, 10]) # set x tick positions
```

```
plt.yticks([-4, -2, 0, 2, 4])          # set y tick positions
plt.xticks(rotation=45)                # rotate x labels

# Add annotations
plt.annotate('Peak', xy=(1.57, 1),
            xytext=(2.5, 0.8),
            arrowprops=dict(arrowstyle='->'))

# Add horizontal / vertical reference lines
plt.axhline(y=0, color='black', linewidth=0.8) # horizontal
plt.axvline(x=np.pi, color='grey', linestyle='--') # vertical
```

Plotting Multiple Lines

```
years    = [2018, 2019, 2020, 2021, 2022, 2023]
sales_A  = [120, 145, 130, 160, 175, 195]
sales_B  = [90, 105, 98, 130, 142, 165]

plt.figure(figsize=(9, 5))
plt.plot(years, sales_A, marker='o', label='Product A', color='steelblue')
plt.plot(years, sales_B, marker='s', label='Product B', color='tomato')
plt.title('Annual Sales Comparison')
plt.xlabel('Year')
plt.ylabel('Sales (units)')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

Lecture 4: Scatter Plot

Scatter plots show the relationship between two continuous variables. Each point represents one observation. Ideal for spotting correlations, clusters, and outliers.

Basic Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
x = np.random.randn(100)
y = 2 * x + np.random.randn(100) * 0.5 # positive correlation

plt.figure(figsize=(7, 5))
plt.scatter(x, y, color='steelblue', alpha=0.7, edgecolors='white', s=60)
plt.title('Scatter Plot - Positive Correlation')
plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True, linestyle='--', alpha=0.4)
plt.show()
```

Scatter Plot Parameters

Parameter	Description
s	Marker size (scalar or array — vary by data value)
c	Colour (string, array, or colormap-mapped values)
cmap	Colormap when c is numeric: 'viridis' 'plasma' 'coolwarm'
alpha	Transparency 0–1 (helps with overlapping points)
edgecolors	Border colour of each marker
marker	Shape: 'o' '^' 's' 'D' '*'

Colour-Mapped & Size-Mapped Scatter

```
# Bubble chart — size and colour encode extra dimensions
np.random.seed(10)
n = 50
x = np.random.rand(n)
y = np.random.rand(n)
sizes = np.random.rand(n) * 500 # bubble size
colors = np.random.rand(n) # colour value

plt.figure(figsize=(8, 6))
sc = plt.scatter(x, y, s=sizes, c=colors, cmap='viridis', alpha=0.7)
plt.colorbar(sc, label='Colour Scale') # add colour bar
plt.title('Bubble Chart')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

Adding a Trend Line

```
# Fit and plot a linear trend line
m, b = np.polyfit(x, y, 1) # degree-1 polynomial (line)
trend = m * x + b

plt.scatter(x, y, alpha=0.6, label='Data')
plt.plot(np.sort(x), m * np.sort(x) + b,
         color='red', linewidth=2, label=f'y = {m:.2f}x + {b:.2f}')
plt.legend()
plt.title('Scatter with Trend Line')
plt.show()
```

Lecture 5: Getting Started with Bar Plot

Bar plots compare discrete categories. Each bar's height (or length) represents a value. Best used when the x-axis is categorical, not continuous.

Basic Vertical Bar Plot

```
categories = ['Python', 'JavaScript', 'Java', 'C++', 'Go']
popularity = [30.3, 13.5, 17.8, 10.2, 5.1]

plt.figure(figsize=(8, 5))
```

```

bars = plt.bar(categories, popularity, color='steelblue', edgecolor='navy',
width=0.6)

# Add value labels on top of each bar
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height + 0.3,
             f'{height}%', ha='center', va='bottom', fontsize=10)

plt.title('Programming Language Popularity')
plt.xlabel('Language')
plt.ylabel('Popularity (%)')
plt.ylim(0, 35)
plt.tight_layout()
plt.show()

```

Bar Plot Parameters

Parameter	Description
width	Width of bars (0–1, default 0.8)
color	Fill colour — string, list, or hex
edgecolor	Border colour of bars
align	'center' (default) 'edge'
bottom	Y value where bar starts — used for stacked bars
label	Label for legend

□ When to use Bar vs Line

Bar plot: comparing discrete, unordered categories (languages, countries, departments).

Line plot: showing a trend over ordered/continuous data (months, years, temperature over time).

Lecture 6: Horizontal, Multiple & Stacked Bars

Horizontal Bar Plot

Use horizontal bars when category names are long or when comparing rankings feels more natural left-to-right.

```

countries = ['India', 'China', 'USA', 'Brazil', 'Germany']
gdp       = [3.75, 17.7, 25.5, 2.1, 4.3]

plt.figure(figsize=(8, 5))
plt.barh(countries, gdp, color='teal', edgecolor='black')
plt.xlabel('GDP (Trillion USD)')
plt.title('GDP by Country')
plt.tight_layout()
plt.show()

```

Multiple (Grouped) Bar Plot

Group multiple bars side-by-side to compare sub-categories across the same groups:

```
import numpy as np

subjects = ['Maths', 'Science', 'English', 'History']
student_A = [85, 92, 78, 88]
student_B = [76, 84, 90, 72]
student_C = [91, 88, 82, 95]

x = np.arange(len(subjects))
width = 0.25

plt.figure(figsize=(9, 5))
plt.bar(x - width, student_A, width, label='Alice', color='steelblue')
plt.bar(x, student_B, width, label='Bob', color='tomato')
plt.bar(x + width, student_C, width, label='Charlie', color='seagreen')

plt.xticks(x, subjects)
plt.xlabel('Subject')
plt.ylabel('Score')
plt.title('Student Score Comparison')
plt.legend()
plt.tight_layout()
plt.show()
```

Stacked Bar Plot

Stack bars to show both the total and the composition of each category:

```
quarters = ['Q1', 'Q2', 'Q3', 'Q4']
product_A = [20, 25, 30, 28]
product_B = [15, 18, 22, 20]
product_C = [10, 12, 14, 18]

plt.figure(figsize=(8, 5))
plt.bar(quarters, product_A, label='Product A', color='steelblue')
plt.bar(quarters, product_B, label='Product B', color='tomato',
        bottom=product_A)

# For third layer, bottom = A + B
bottom_AB = [a+b for a,b in zip(product_A, product_B)]
plt.bar(quarters, product_C, label='Product C', color='seagreen',
        bottom=bottom_AB)

plt.title('Quarterly Sales by Product (Stacked)')
plt.xlabel('Quarter')
plt.ylabel('Sales')
plt.legend()
plt.tight_layout()
plt.show()
```

Lecture 7: Histogram

A histogram shows the distribution of a continuous variable by grouping values into bins and counting how many fall in each bin. It answers: 'what does the data look like?'

□ Bar chart vs Histogram

Bar chart: categories on x-axis (discrete, unordered).
 Histogram: continuous numeric data on x-axis, grouped into bins.
 No gaps between bars in a histogram (continuous data).

Basic Histogram

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
data = np.random.randn(1000)    # 1000 values from normal distribution

plt.figure(figsize=(8, 5))
plt.hist(data, bins=30, color='steelblue', edgecolor='white', alpha=0.8)
plt.title('Normal Distribution - 1000 Samples')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True, linestyle='--', alpha=0.4)
plt.show()
```

Histogram Parameters

Parameter	Description
bins	Number of bins (int) or bin edges (list). Default: 10
range	(min, max) — limit data range included
density	True → y-axis shows probability density instead of count
cumulative	True → each bar shows running total
color	Fill colour of bars
edgecolor	Border colour (use 'white' to visually separate bins)
alpha	Transparency — useful when overlapping histograms
label	Legend label

Overlapping Histograms & KDE

```
group_A = np.random.normal(60, 10, 500)    # mean=60, std=10
group_B = np.random.normal(75, 8, 500)     # mean=75, std=8

plt.figure(figsize=(9, 5))
plt.hist(group_A, bins=30, alpha=0.6, color='steelblue', label='Group A')
plt.hist(group_B, bins=30, alpha=0.6, color='tomato', label='Group B')
plt.title('Score Distribution: Group A vs Group B')
plt.xlabel('Score')
plt.ylabel('Count')
plt.legend()
plt.show()

# Add KDE curve (requires scipy)
from scipy.stats import gaussian_kde
kde = gaussian_kde(group_A)
x_range = np.linspace(group_A.min(), group_A.max(), 200)
```

```
plt.plot(x_range, kde(x_range) * len(group_A) * (group_A.max() - group_A.min()) / 30,
        'navy', linewidth=2, label='KDE')
```

Choosing the Right Number of Bins

Rule	Formula / Guidance
Sturges	$\text{bins} = 1 + \log_2(n)$ — good for symmetric data
Square root	$\text{bins} = \sqrt{n}$ — simple, general purpose
Auto (NumPy)	<code>np.histogram_bin_edges(data, bins='auto')</code> — picks best
Too few bins	Hides the shape — distribution looks flat
Too many bins	Noisy — every bar is 1–2 observations

Lecture 8: Pie Chart & Changing Styles

Basic Pie Chart

```
labels = ['Python', 'JavaScript', 'Java', 'C++', 'Others']
sizes = [35, 25, 20, 12, 8]
explode = [0.05, 0, 0, 0, 0] # slightly pull out first slice
colors = ['steelblue', 'tomato', 'seagreen', 'gold', 'orchid']

plt.figure(figsize=(7, 7))
plt.pie(sizes,
        labels=labels,
        autopct='%1.1f%%', # show percentages
        explode=explode,
        colors=colors,
        shadow=True,
        startangle=140) # rotate start position

plt.title('Programming Language Share')
plt.axis('equal') # keep circle (not ellipse)
plt.show()
```

Pie Chart Parameters

Parameter	Description
<code>autopct</code>	Format string for % labels: '%1.1f%%' shows e.g. 35.0%
<code>explode</code>	List of offsets — 0.1 pulls that slice outward
<code>startangle</code>	Rotation of first slice from positive x-axis
<code>shadow</code>	True adds a drop shadow
<code>colors</code>	List of colours for slices
<code>wedgeprops</code>	Dict of wedge style: {'edgecolor': 'white', 'linewidth': 2}
<code>pctdistance</code>	Distance of % label from centre (default 0.6)

❑ When NOT to use pie charts

Avoid when comparing more than 5–6 slices — becomes hard to read.

Avoid when slices are close in size — a bar chart shows differences far more clearly.

Use a bar chart when you need precision; use pie only for rough proportions.

Changing Plot Styles

Matplotlib provides built-in stylesheets that change colours, fonts, gridlines, and backgrounds with one line:

```
# List all available styles
print(plt.style.available)

# Apply a style
plt.style.use('seaborn-v0_8-darkgrid')    # clean dark grid
plt.style.use('ggplot')                  # R ggplot2 look
plt.style.use('fivethirtyeight')         # FiveThirtyEight style
plt.style.use('dark_background')         # dark theme
plt.style.use('bmh')                     # Bayesian methods style

# Use context manager for temporary style
with plt.style.context('seaborn-v0_8-whitegrid'):
    plt.plot([1,2,3], [4,5,6])
    plt.show()

# Style resets after the with block
```

Manual Style Customisation

```
# Set global defaults via rcParams
plt.rcParams['figure.figsize'] = (9, 5)
plt.rcParams['font.size'] = 12
plt.rcParams['font.family'] = 'Arial'
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['lines.linewidth'] = 2
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.linestyle'] = '--'
plt.rcParams['grid.alpha'] = 0.5

# Reset to defaults
plt.rcParams.update(plt.rcParamsDefault)
```

End of Module 9 Notes — Generative AI Course